

MATEMÁTICA

UNIDAD N°2

TRABAJO INTEGRADOR

2DA PARTE

MATEMÁTICA

TRABAJO PRÁCTICO INTEGRADOR - UNIDAD 2

Álgebra de Boole y Lógica Digital: Modelado de Sistemas con R

1. El Salto de la Teoría al Silicio

En la Unidad 1 validamos razonamientos; en esta Unidad 2, utilizaremos **R** y **RStudio** para diseñar y optimizar "hardware lógico". En programación de bajo nivel y arquitectura de sistemas, el Álgebra de Boole es la herramienta que permite que los procesadores tomen decisiones mediante miles de millones de interruptores. En R, seguiremos operando con `TRUE` (1 lógico) y `FALSE` (0 lógico).

2. Diccionario de Equivalencias Digitales

Para este TP, traduciremos los conceptos de los circuitos y mapas de Karnaugh a operadores de código que R procesa de forma vectorial:

- **Compuerta AND (Producto):** Se usa `&`. Representa la conexión en serie. Solo es `TRUE` si todas las entradas son `TRUE`.
- **Compuerta OR (Suma):** Se usa `|`. Representa la conexión en paralelo. Es `TRUE` si al menos una entrada es `TRUE`.
- **Compuerta NOT (Inversor):** Se usa `!`. Invierte el estado lógico.
- **Compuerta XOR (OR Exclusiva):** Se usa `xor(a, b)`. Es `TRUE` solo si las entradas son distintas.
- **Compuerta NAND / NOR:** Son las negaciones de las básicas: `!(a & b)` y `!(a | b)`.

Guía de Implementación Booleana en R

Concepto Digital	Operador en R	Ejemplo de Código
Producto Lógico (AND)	<code>&</code>	<code>A & B</code>
Suma Lógica (OR)	<code> </code>	<code>A B</code>
Inversor (NOT)	<code>!</code>	<code>!A</code>
OR Exclusiva (XOR)	<code>xor()</code>	<code>xor(A, B)</code>
Universal NAND	<code>!(&)</code>	<code>!(A & B)</code>
Universal NOR	<code>!()</code>	<code>!(A B)</code>
Igualdad (XNOR)	<code>==</code>	<code>A == B</code>

MATEMÁTICA

3. Los Tres Desafíos Integradores

Desafío 1: Generador Automático de Formas Canónicas (SOP)

Contexto: En lugar de buscar los unos manualmente en una tabla de papel, usaremos R para identificar los **mintérminos** de un sistema de 3 variables (A, B, C).

Instrucciones:

1. Crea un `data.frame` que contenga todas las combinaciones posibles (8 filas) para las variables A, B y C.
2. Define una columna `Salida` basada en una regla arbitraria, por ejemplo: `Salida <- (A | B) & !C`.
3. **Tu tarea:** Utiliza funciones de filtrado de R para mostrar únicamente las filas donde `Salida == TRUE`.
4. Escribe en un comentario del script la expresión **SOP (Suma de Productos)** canónica resultante de esos mintérminos (Ejemplo: `m2 + m4...`).

Desafío 2: El Desafío de la Universalidad (NAND)

Contexto: En la industria, a veces solo se dispone de un tipo de compuerta. Debes demostrar la **Universalidad de la NAND**.

Instrucciones:

1. Crea una función propia en R llamada `nand_gate <- function(a, b) { !(a & b) }`.
2. **Tu misión:** Utilizando **únicamente** la función `nand_gate`, debes construir una estructura que se comporte exactamente como una compuerta **OR**.
3. **Prueba obligatoria:** Genera una tabla de verdad comparando el resultado de tu "OR hecho con NANDs" contra el operador `|` de R. Deben ser idénticos.

Desafío 3: El "Kill Switch" y Minimización de Reglas de Negocio

Contexto: Vas a programar el sistema de aprobación de créditos bancarios visto en el PDF (pág. 16).

Instrucciones:

1. Define las variables: `ingresos`, `antiguedad`, `vip` y `deuda` con valores lógicos.
2. Programa la función original: `F_original <- ((ingresos & antiguedad) | vip) & !deuda`.
3. **El Problema:** Imagina que mediante un **Mapa de Karnaugh** descubres que para un segmento específico la función se simplifica a `F_minima <- ingresos & !deuda`.

MATEMÁTICA

4. **Tu misión:** Crea un vector de prueba con 10 casos diferentes y usa la función `identical(F_original, F_minima)` para verificar si la simplificación es válida o si se perdió lógica en el camino. Explica en los comentarios por qué la variable `deuda` actúa como un "Kill Switch".
-

4. ¿Qué se evalúa? (Rúbrica de éxito)

- **Rigor Técnico:** Uso correcto de operadores booleanos en R y paréntesis para jerarquizar operaciones.
- **Capacidad de Síntesis:** Identificación correcta de minterminos y su traducción a expresiones SOP.
- **Optimización:** Demostración lógica de la equivalencia entre circuitos complejos y versiones simplificadas.
- **Documentación:** El script debe explicar qué leyes del Álgebra de Boole (Idempotencia, De Morgan, Absorción, etc.) justifican cada paso de tu código.